

Task: Document Ingestion Pipeline



Overview

In this task, your objective is to debug, fix, and successfully execute the Jupyter notebook:

- `Day1_Task1_Document_Ingestion.ipynb`

The notebook contains intentional issues (such as broken import statements and incomplete code cells) that prevent it from executing properly. Your mission is to fix these bugs and ensure the end-to-end document ingestion pipeline runs seamlessly.



Objectives

By completing this task, you will:

1. **Understand Clinical Grounding:** Learn why grounding against verified clinical documents is critical for eliminating LLM hallucinations.
 2. **Fix Parsing & Imports:** Debug and resolve broken imports and loading logic for document loaders (e.g., `PyPDFLoader`).
 3. **Normalize Metadata:** Ensure proper citation metadata (document names, 1-indexed page numbers) is attached to document chunks.
 4. **Evaluate Chunking Strategies:** Compare naive fixed-size splitting against section-aware recursive splitting.
 5. **Generate Embeddings:** Compute embeddings and measure semantic cosine similarity across related and unrelated clinical phrases.
 6. **Build & Query Vector Store:** Index the processed chunks into a vector database and retrieve relevant guideline passages with citation scores.
-



Step-by-Step Task Guide

1. Setup & Environment

- Open `Day1_Task1_Document_Ingestion.ipynb`.
- Run the setup cell and verify paths and configuration parameters.

2. Identify & Fix Broken Code Cells

- Inspect the document loading cells. Identify incomplete statements (such as missing module imports in `from langchain_community.document_loaders...`).
- Correct the syntax and ensure all dependencies load cleanly.

3. Parse Document & Inspect Metadata

- Load document pages and verify raw vs. normalized metadata structures.
- Validate that headings and clinical text are cleanly extracted without formatting artifacts.

4. Implement & Compare Chunking Strategies

- Execute both naive fixed-character chunking and section-aware recursive chunking.
- Observe chunk boundaries to ensure semantic integrity is preserved at sentence/paragraph levels.

5. Validate Embeddings & Semantic Similarity

- Generate vector embeddings for sample clinical statements.
- Verify cosine similarity outputs: semantically related statements should score significantly higher than unrelated queries.

6. Build the Vector Index & Run Query

- Build and persist the vector database index from the chunked documents.
- Execute the clinical query and verify that top-ranked chunks return accurate text, high relevance scores, and complete citation metadata.



Acceptance Criteria (Definition of Done)

- ☐ All cells in `Day1_Task1_Document_Ingestion.ipynb` execute sequentially from top to bottom without errors.
- ☐ All broken import statements and syntax errors are identified and corrected.
- ☐ Chunking comparison clearly demonstrates section-aware boundaries vs. naive fixed-size cuts.
- ☐ Cosine similarity test confirms higher semantic affinity for equivalent medical concepts.
- ☐ Final retrieval query returns top relevant passages complete with document name, page number, and score.